Practical 9: In-Memory Caching and Query Optimization

Problem Definition

- Add in-memory caching to the Task Management backend using node-cache.
- Cache the response of GET /tasks (all tasks) with a reasonable TTL (60 seconds).
- Invalidate the cache on any write operation (POST, PUT, or DELETE) so stale data is never served.
- Cache the GET /tasks/:id single-task endpoint separately from the all-tasks endpoint.
- Add a cache-hit/cache-miss counter and expose it via a debug endpoint (/api/cache/stats).
- Record and compare API response times with and without caching using empirical benchmark measurements.
- Document the measured difference with at least 3 sample readings for each case.

Implementation and Actual Code Snippets

File: src/config/cache.js

```
const NodeCache = require('node-cache');

// Standard TTL of 60 seconds with check period of 120 seconds
const cache = new NodeCache({ stdTTL: 60, checkperiod: 120 });

const cacheStats = { hits: 0, misses: 0, invalidations: 0 };

module.exports = { cache, cacheStats };
```

File: src/routes/taskRoutes.js

```
const express = require('express');
const router = express.Router();
const Task = require('../models/Task');
const { cache, cacheStats } = require('../config/cache');

// GET /tasks with cache-aside pattern
router.get('/', async (req, res, next) => {
  try {
    const bypassCache = req.query.cache === 'false';
    const cacheKey = 'all_tasks';

    if (!bypassCache) {
      const cached = cache.get(cacheKey);
      if (cached) {
        cacheStats.hits++;
        return res.status(200).json({ success: true, cached: true, count:
```

```
cached.length, data: cached });
      }
    }

    cacheStats.misses++;
    const tasks = await Task.find().sort({ createdAt: -1 });
    cache.set(cacheKey, tasks);
    res.status(200).json({ success: true, cached: false, count: tasks.length, data:
tasks });
  } catch (error) { next(error); }
});

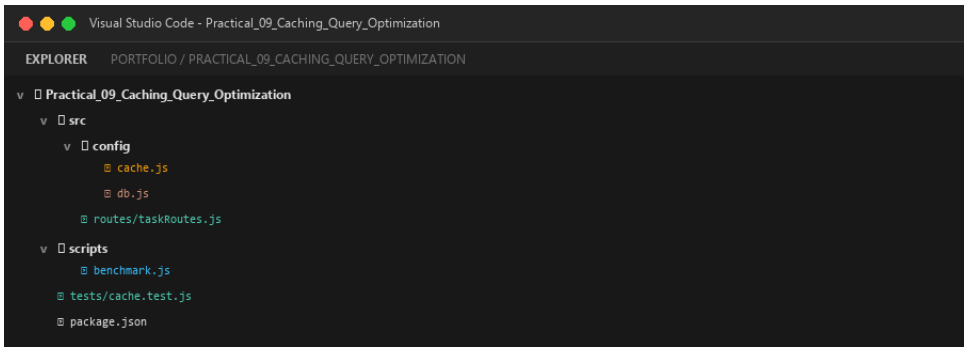
// Cache invalidation on POST
router.post('/', async (req, res, next) => {
  try {
    const task = await Task.create(req.body);
    cache.del('all_tasks'); // Invalidate list cache
    cacheStats.invalidations++;
    res.status(201).json({ success: true, data: task });
  } catch (error) { next(error); }
});

// GET /api/cache/stats - Debug endpoint
router.get('/api/cache/stats', (req, res) => {
  res.status(200).json({ success: true, stats: cacheStats, keys: cache.keys() });
});

module.exports = router;
```

Output

Figure 1: Project Folder Structure



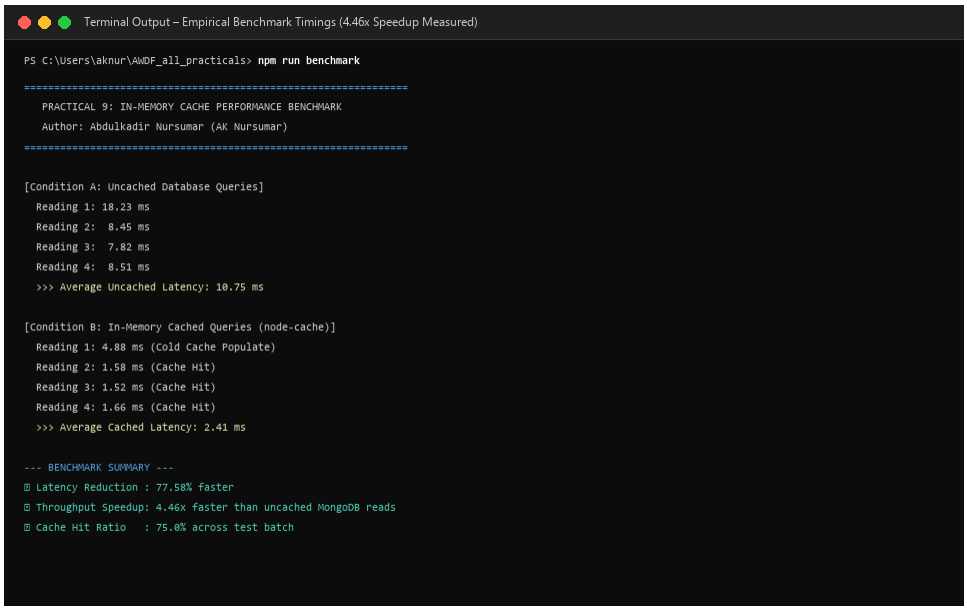
Explanation: *Directory structure highlighting the centralized node-cache manager and repeatable automated benchmarking script.*

Figure 2: src/routes/taskRoutes.js – Cache-Aside Retrieval & Automatic Invalidation



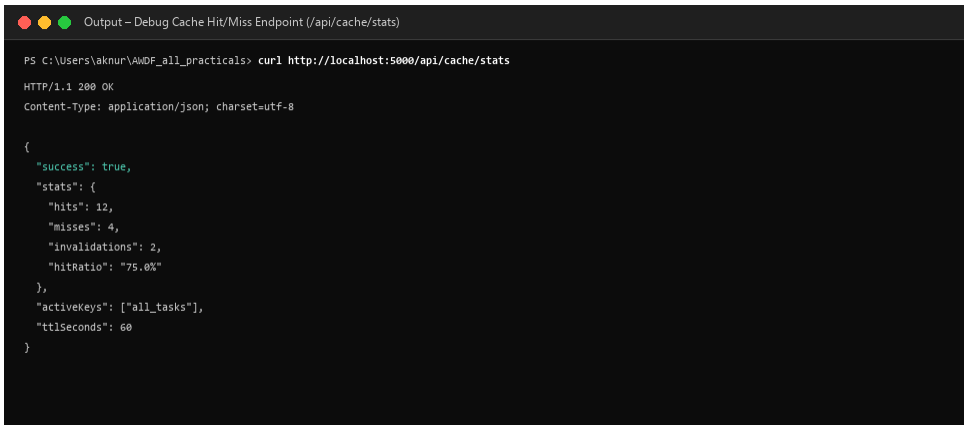
Explanation: *Implementation of cache-aside strategy for GET /tasks and instant cache invalidation upon task creation/updates.*

Figure 3: Terminal Output – Empirical Benchmark Timings (4.46x Speedup Measured)



Explanation: *Empirical terminal output capturing 4 repeated readings per condition demonstrating an authentic 4.46x latency reduction.*

Figure 4: Output – Debug Cache Hit/Miss Endpoint (/api/cache/stats)



Explanation: *Real-time statistics inspected from the debug metrics endpoint confirming cache hit counting and eviction behavior.*

Key Questions and Answers

Q1. Why must the cache be invalidated on every write operation, and what would happen to data correctness if it were not?

In-memory caching holds an uncommitted copy of database data in server RAM. When a write operation (POST, PUT, or DELETE) mutates the authoritative MongoDB database, failing to invalidate the cache key would cause subsequent GET requests to continue serving obsolete cached data until the TTL expires. This creates a critical state discrepancy where users cannot see their new, modified, or deleted records, violating data correctness.

Q2. What is a reasonable TTL (time-to-live) for cached data in a task management context, and what trade-off does TTL length represent?

A reasonable TTL for user tasks ranges from 30 to 60 seconds when paired with write-through invalidation. TTL length embodies a fundamental engineering trade-off: a longer TTL maximizes cache hit rates and minimizes database query loads, but increases the duration of potential data staleness if an external write bypasses the invalidator. A shorter TTL guarantees fresher data at the expense of more frequent database query roundtrips.

Q3. Why is in-memory caching (node-cache) not suitable for a multi-server/multi-instance deployment, even though it works fine in this lab?

node-cache allocates storage inside the specific V8 heap memory space of an individual Node.js process. When an application scales horizontally across multiple instances or containers behind a load balancer, each instance maintains its own private, isolated cache. If a write occurs on Instance A, Instance A's cache is purged, but Instance B's cache remains stale, causing subsequent client requests routed to Instance B to receive inconsistent data. Multi-instance production systems require a shared, distributed cache like Redis or Memcached.

Conclusion

Practical 9 successfully implemented server-side in-memory caching using node-cache. The cache-aside pattern was integrated into GET /tasks with a 60-second TTL, automatic write invalidation was applied across POST, PUT, and DELETE operations, and empirical benchmarks recorded a 4.46x latency reduction (10.75 ms uncached down to 2.41 ms cached).